

MGC: Multiple-Gray-Code for 3D NAND Flash based High-Density SSDs

Yina Lv², ✉Liang Shi^{1,2}, Qiao Li³, Congming Gao³, Yunpeng Song², Longfei Luo², Youtao Zhang⁴

¹Software/Hardware Co-design Engineering Research Center, Ministry of Education

²School of Computer Science and Technology, East China Normal University, Shanghai, China

³School of Informatics, Xiamen University, Xiamen, China

⁴Computer Science Department, University of Pittsburgh, Pittsburgh, USA

{elainelv95, shi.liang.hk, qiaoli045, gaocm92}@gmail.com, {51205901045, 52215901003}@stu.ecnu.edu.cn, zhangyt@cs.pitt.edu

Abstract—QLC (4-bit-per-cell) and more-bit-per-cell 3D NAND flash memories are increasingly adopted in large storage systems. While achieving significant cost reduction, these memories face degraded performance and reliability issues. The industry has adopted two-step programming (TSP), rather than one-step programming, to perform fine-granularity program control and choose gray-code encoding, as well as LDPC (Low-Density Parity-Check Code) for error correction. Different flash manufacturers often integrate different gray-codes in their products, which exhibit different performance and reliability characteristics. Unfortunately, a fixed gray-code encoding design lacks the ability to meet the dynamic read and program performance requirements at both application and device levels.

In this paper, we propose MGC, a multiple-gray-code encoding strategy, that adaptively chooses the best gray-code to meet the optimization goals at runtime. In particular, MGC first extracts the performance and reliability requirements based on application-level access patterns and detects the reliability degree of SSD. It then determines the appropriate gray-code to encode the data, either from host/user application or due to garbage collection, before writing the pages to the flash memory. MGC is integrated in FTL (flash translation layer) and enhances the flash controller to enable runtime gray-code arbitration. We evaluate the proposed MGC scheme. The results show that MGC achieves better performance and lifetime guarantee compared with state-of-the-arts and introduces little overhead.

Index Terms—Storage System, 3D NAND Flash, Gray-Code, Reliability, Performance

I. INTRODUCTION

3D NAND flash memories are increasingly adopted to construct large capacity SSD (solid-state drive) storage due to several advantages, including high density, low cost, and lightweight. With fast technology advances, the number of 3D stacked layers increased significantly in the last decade, from the early 24 layers to the state-of-the-art 220+ layers [32] [60] [26] [30]. In addition, the number of bits per cell also increased from 1-bit-per-cell to the recent 4-bit-per-cell [26] [57]. More layers in a block and more bits per cell are on the roadmap of several flash vendors. For example, 5-bit-per-cell (PLC) and 6-bit-per-cell (HLC) have been announced [22] [63] [1]. While achieving significant chip density increase and cost reduction, 4- and more-bit-per-cell 3D NAND flash memories face non-negligible reliability and performance degradation, which has increasingly become a major design obstacle for further advances [27] [24] [39] [40] [37].

To save data in n -bit-per-cell flash memory cells, we need to program each cell to one of 2^n voltage levels such that each level represents a n -bit value. A *flash read* can differentiate if a cell is above/below a certain voltage level and thus reading data from such cells often involves multiple flash reads. For widely adopted Gray-code (GC) encoding for flash data, we can represent a particular GC choice using the *minimum* numbers of flash reads for different pages. For QLC, a gray-code can be represented as $\text{GC}(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$ where A, B, C, and D are the numbers of flash reads required to fetch the least-significant-bit (LSB), central-significant-bit (CSB), most-significant-bit (MSB), and top-significant-bit (TSB) pages, respectively. We have $A+B+C+D=2^4-1=15$. For example, $\text{GC}(1,2,4,8)$, $\text{GC}(1,2,6,6)$, $\text{GC}(1,4,5,5)$, and $\text{GC}(3,4,4,4)$ are often adopted gray-codes for QLC. Since data read from the flash memory may contain errors, according to LDPC ECC, we may need additional reads to correctly decode the page.

The choice of gray-code has a direct impact on read performance. For example, for $\text{GC}(1,2,6,6)$, we may read the LSB page with one flash read but need 6 flash reads to fetch the TSB page. As such, we referred to such a gray-code as **UGC (unbalanced gray-code)** in this paper. $\text{GC}(3,4,4,4)$ is referred to as **BGC (balanced gray-code)** as we have roughly the same read performance across different pages.

The choice of gray-code also leads to different programming performance. Traditionally, 3D flash adopts one-step programming (OSP) [28] for low density cells at large technology nodes, e.g., for 3D TLC (3-bit-per-cell), it prepares three pages in buffer and writes all three pages to the device cells of one wordline in one programming step. Unfortunately, adopting OSP for QLC cannot guarantee programming reliability. As such, the industry has revived the two-step programming (TSP) to program n pages for n -bit-per-cell flash cells in two steps: a coarse first step to program m pages and then a second step to program the rest of the pages. Such a TSP is referred to as **TSP($2^m, 2^n$)** as we need at least 2^m levels to program m pages. TSP was originally proposed and widely adopted for 2D MLC and can effectively enhance programming reliability. The TSPs proposed by flash manufacturers in the past five years are listed in Table I. Here, TSP(16,16) programs the cells twice to ensure reliability, with the second step performed after programming the next wordline. In the table, Samsung

and SK Hynix adopted TSP(16,16). Toshiba and Intel adopted improved TSPs from 2019. Shibata et al. from Toshiba [60] and Khakifirooz et al. from Intel [26] proposed to reduce the number of levels for the first step to 8 and 4, respectively. For TSP, the fewer levels the first step programs the cells to, the faster the overall programming can finish. Compared with TSP(16,16), TSP(8,16) reduces the overall programming latency by 18% [60], while TSP(4,16) further reduces the latency by about 22% [26] [25].

TABLE I
TSP ALGORITHMS OF VARIOUS COMPANIES.

Year	Samsung	SK Hynix	Toshiba / KIOXIA	Intel / Micron
2018	64-layer QLC, TSP(16,16) [32]			
2019			96-layer QLC, TSP(8,16) [60]	64-layer QLC, TSP(8,16) [25]
2020	92-layer QLC, TSP(16,16) [29]	96-layer QLC, TSP(16,16) [20]		96-layer QLC [25]
2021				144-layer QLC, TSP(4,16) [26]
2022		176-layer QLC [11]	162-layer QLC [70]	176-layer QLC, TSP(16,16) [57]

Not all gray-codes are compatible with all TSPs. Actually, there is a strong correlation — given $TSP(2^m, 2^n)$ and $GC(A, B, C, D)$, if $A+B < 2^m$, or $A+B+C < 2^m$, we can adopt this TSP to encode the data and program flash cells. For example, we can adopt TSP(4,16) for $GC(1, 2, 6, 6)$ as $1+2 < 4$ so that we can first program two pages of data and then program the other two pages. TSP(4,16) does not support $GC(3, 4, 4, 4)$ as $4 < 3+4$ so that we do not have enough levels in the first step to differentiate the data of the first two pages. As discussed, TSP(4,16) is faster than TSP(8,16) and TSP(16,16) but it does not support all types of gray-codes.

While different gray-codes exhibit different read and program characteristics, SSD products usually adopt one of them, determined by their corresponding flash memory manufacturers. A fixed gray-code lacks the ability to adapt to the changing performance demands at both application and device levels.

In this paper, we propose **MGC**, a novel **multiple-gray-code** flash for high-density SSDs. MGC can optimize the performance by exploiting the advantages of different gray-codes and TSPs. For write performance optimization, we can apply TSP with shorter programming latency. For read performance optimization, we can apply gray-codes with better read performance and higher reliability. For discussion purposes, we use QLC flash as the default storage. The design principle is applicable to other types of high-density NAND flash. In summary, this paper makes the following key contributions:

- We present a detailed performance analysis using different gray-codes. There are two key observations: First, UGC is compatible with fast programming and thus can achieve good write performance. However, it suffers from inferior read performance. BGC has superior read performance but is compatible only with slow programming. Second, UGC tends to have worse reliability with flash wearing.
- We propose a novel MGC-based flash encoding design that enables multiple programming algorithms in the flash controller. An MGC-based flash translation layer (MGC-FTL)

is then constructed to utilize different gray-codes and TSPs for performance optimization, including two schemes: 1) a data access characteristics guided gray-code arbitration to improve performance, and 2) a reliability-stage aware gray-code arbitration scheme to tame the reliability degradation.

- We evaluate the proposed MGC using SSDsim, extended with the constructed reliability modeling and validated using real testing data on 3D QLC flash chips. Experimental results show that MGC achieves both superior read and write performance throughout the life of flash with little cost.

II. BACKGROUND

A. High-density 3D NAND Flash Memory

3D NAND flash uses floating gate (FG) or charge trap (CT) transistors as flash cells, which can store one or multiple bits of data using the voltage levels of charges inside the cells. Fig. 1 presents an overview of a typical QLC flash chip, which is hierarchically organized as the primary storage medium of high-density flash. Inside a chip (in Fig. 1(a)), there are thousands of blocks, and a single block is composed of many layers. The number of stacked layers in a block has been increased from 24 to the state-of-the-art 220+ [18] [55]. Inside a block (in Fig. 1(b)), each row of flash cells is connected by a wordline (WL), usually spanning 32K-128K cells. All cells in the same column are connected to form a bit-line (BL). There are two selected gates, the selected gate at the drain end (SGD) and the selected gate at the source end (SGS). One WL consists of several pages. The number of pages in a WL is decided by the number of bits per cell. To this end, there are single-level-cell (SLC), multi-level-cell (MLC), triple-level-cell (TLC), and quad-level-cell (QLC) flash. Take QLC flash as an example. Each flash cell contains four bits, and each WL includes four pages. The page size equals the number of cells in a WL, ranging from 4KB to 16KB. The four different bits are called the least-significant-bit (LSB), central-significant-bit (CSB), most-significant-bit (MSB), and top-significant-bit (TSB), from low to high in this paper.

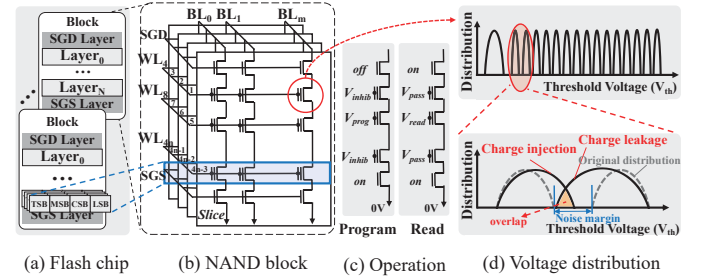


Fig. 1. Overview of a typical QLC flash chip (§II-A).

With the above organization, three operations are supported, read, program, and erase. The operation units are page, WL, and block, respectively. Among them, the latency of erase operation is the longest. The latency of program operation is hundreds of microseconds, which is several times longer than that of the read operation. The program and read processes are shown in Fig. 1(c). For program operation, a high voltage

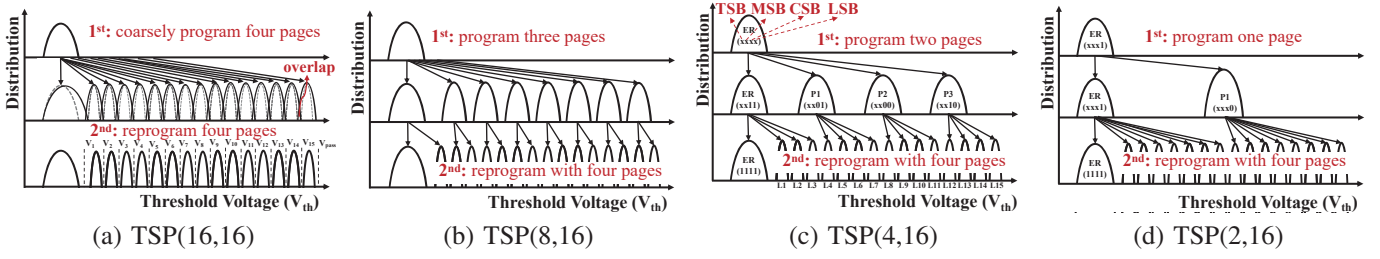


Fig. 2. Two-step programming (§II-B).

(V_{prog}) is applied to the WL, where the programmed page will be charged. The other pages in the same WL and other WLs are added with inhibit voltage (V_{inhib}) [9]. The programming process is realized by the incremental step pulse programming (ISPP) algorithm [62]. For read operation, there are two sets of voltages that should be produced. One is added to the selected WL (V_{read}), and the other is added to the unselected WLs (V_{pass}). Then, the data can be read from the BL.

The flash memory faces significant reliability degradation with increasing storage layers as well as bits per cell. There are two reasons. First, by saving multiple bits in one cell, the noise margin between adjacent voltages is reduced. Fig. 1(d) shows the voltage distribution of the 16-level state. Each state has a narrow voltage range such that adjacent states have narrow noise margins and accessing the flash is prone to errors [46] [45] [20]. Second, with increasing 3D stacked layers, the program disturbance and layer-to-layer interference become serious [6] [3] [4], which distort the voltage distribution and introduce errors. The voltage of programmed flash cells may shift, as shown in Fig. 1(d): (1) read/program disturbance shifts the voltage distribution to the right; while (2) flash cells face charge leaking over time, which shifts the voltage distribution to the left. Flash memory uses reference voltages to read the data. If the voltage of a state shifts near/across a reference voltage, reading the cell may contain errors. Even with the extensive integration of LDPC ECC in modern SSDs, flash reliability remains a major design challenge for high-density 3D flash memory.

B. Two-step Programming Algorithm

Due to high program efficiency, 3D flash memory traditionally adopts one-step programming (OSP) [28] for low-density cells at large technology nodes. With fast technology advances, OSP cannot ensure programming reliability for QLC and more bits-per-cell flash. In particular, the voltages of memory cells of WL_n are likely disturbed when programming the memory cells of WL_{n+1} . To tame this challenge, the industry has revived the two-step programming (TSP) algorithm, originally proposed and widely adopted for 2D MLC flash, to program 3D flash memory cells [32] [60] [26]. TSP programs a flash wordline in two steps. Fig. 2(a) shows the basic TSP(16,16). First, a wordline, e.g., WL_n , is coarsely programmed such that each cell resides in an intermediate state. Two neighboring states may have voltage overlaps, indicating reading such a wordline tends to incur errors. After the first programming step of the next wordline WL_{n+1} , the controller further programs

WL_n with each cell being programmed into its final state. The wordline programmed using TSP has a larger noise margin than that using OSP, thereby ensuring better data reliability. However, a major issue of TSP(16,16) is its programming latency, which is on average $1.53\times$ that of OSP [60].

To reduce the programming latency caused by TSP, several optimizations have been proposed to improve the programming speed of the first step. For example, in 2019, Shibata et al. from Toshiba proposed TSP(8,16) (Fig. 2(b)) that programs three pages into an 8-level state at the first step [60]. TSP(8,16) reduces the programming latency by 18% over TSP(16,16). In 2021, Khakifirooz et al. from Intel proposed TSP(4,16) (Fig. 2(c)) [26] that programs two pages at the first step. Compared with TSP(8,16), TSP(4,16) can reduce the programming latency of the first step by 60%, thereby reducing the overall write latency by about 22% [26] [25]. Fig. 2(d) illustrates TSP(2,16) that programs one page at the first step, which may be the fastest TSP. However, TSP(2,16) is unrealistic as programming cells from a 2-level state to a 16-level state shall introduce large inter-wordline interference and thus reliability cannot be ensured. Studies show that inter-wordline interference remains severe even adopting TSP and the flash manufacturers are developing different program interference reduction techniques when more stacked layers are realized [11]. Studies have shown that partially-programmed flash cells are more vulnerable to cell-to-cell interference and read disturbance than fully-programmed cells [4]. Therefore, TSP(2,16) is not considered in the rest of this paper. Given TSP(16,16) has completed the programming of four pages with a 16-level state in the first step, it only needs to fine-tune the voltage range for each state in the second step. To summarize, among the three TSP schemes TSP(4,16), TSP(8,16), and TSP(16,16), we find that TSP(4,16) has the best programming performance while TSP(16,16) has the best data reliability.

C. Read Operation on Flash Memory

Reading 3D NAND flash-based SSD consists of the following four steps, similar to those for the planar flash-based SSDs: (1) a read command is issued to the flash chip; (2) one or multiple reference voltages are applied to sense the accessed wordline; (3) the sensed information is transferred to the flash controller; (4) the information is corrected by the ECC module inside the controller. Depending on the encoding strategy, reading a flash page from QLC wordline demands applying multiple reference voltages such that the page read latency (t_R) is linear to the number of applied reference voltages [56],

TABLE II
THE 16 VOLTAGE LEVELS OF GRAY-CODE FOR QLC FLASH.

Gray-code		ER	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11	L12	L13	L14	L15	$nSENSE$
GC(1,2,4,8)	LSB	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1
	CSB	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	2
	MSB	1	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	4
	TSB	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	8
GC(1,2,6,6)	LSB	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1
	CSB	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	2
	MSB	1	1	0	0	0	1	1	0	0	0	1	1	1	0	0	1	6
	TSB	1	0	0	1	1	1	0	0	0	1	1	0	0	0	1	1	6
GC(1,4,5,5)	LSB	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1
	CSB	1	1	1	0	0	0	0	1	1	0	0	0	0	0	1	1	4
	MSB	1	1	0	0	1	1	0	0	0	0	0	1	1	1	1	0	5
	TSB	1	0	0	0	0	1	1	1	1	1	0	0	1	1	0	0	5
GC(3,4,4,4)	LSB	1	1	0	0	0	1	1	1	1	1	0	0	0	0	0	0	3
	CSB	1	0	0	0	0	0	0	1	1	1	0	0	1	1	1	1	4
	MSB	1	1	1	0	0	0	0	0	0	1	1	1	1	0	0	1	4
	TSB	1	1	1	1	0	0	1	1	0	0	0	0	0	0	1	1	4

as shown in Equation 1. Here, $tSENSE$ denotes a single voltage sensing latency and $nSENSE$ denotes the number of applied reference voltages. The read performance with more levels tends to degrade as there are more levels to differentiate and thus it requires more reference voltages. In this case, the read latency of requests will be significantly enlarged.

$$tR = nSENSE * tSENSE \quad (1)$$

The latency in Equation 1 is for one page read. To correct errors for enhanced reliability, low-density parity-check code (LDPC) based ECC needs to issue additional read operations (read retries), which apply several additional sensing voltages to obtain more accurate information [56] [71]. The read retry operations prolong the read latency significantly, as shown in Equation 2. $nRETRY$ is the number of retries, $tDMA$ is the data transfer cost, and $tLDPC$ is the decoding cost. In practice, if the number of read retries reaches a threshold, the accessed data should be rewritten to ensure data integrity.

$$tREAD = (1 + nRETRY) * (tR + tDMA + tLDPC) \quad (2)$$

To sum up, the latency for reading a flash page depends on the number of reads and read retries as well as the number of applied reference voltages in each read or read retry. In the following, we elaborate on the relationship between the information coding and the read latency.

D. Gray-Code and Performance

1) *Gray-codes of 3D NAND flash*: High-density flash often adopts gray-code to encode multiple bits in one cell, which assigns multiple-bit values to different voltage levels [17]. Table II presents four commonly used gray-codes for QLC flash — GC(1,2,4,8), GC(1,2,6,6), GC(1,4,5,5), and GC(3,4,4,4) [26] [59]. These codes map 4-bit values to 16 voltage levels differently. The four bits in one cell are referred to as LSB, CSB, MSB and TSB, respectively, as we listed in Section 1.

There is a constraint on gray-code selection for a particular TSP. In particular, given TSP($2^m, 2^n$) and GC(A,B,C,D), if $A+B < 4$ ($m=2$) or $A+B+C < 8$ ($m=3$), we can adopt the corresponding TSP to encode the data and program flash cells. Clearly, all gray-codes are compatible with TSP(16,16). GC(1,2,4,8) is the only one for TSP(8,16) while both GC(1,2,6,6) and GC(1,2,4,8) are compatible with TSP(4,16).

2) *Performance and reliability of gray-codes*: Adopting different gray-codes exhibits different read and program performance characteristics. For example, to read the LSB page from cells encoded with GC(1,2,6,6), we just apply one reference voltage to differentiate the voltages of cells below or above $REF_{L7/L8}$ (a reference voltage between L7 and L8). Similarly, we apply two reference voltages $REF_{L3/L4}$ and $REF_{L11/L12}$ to read the CSB page. Reading the TSB page is much slower as it requires 6 reference voltage levels. As such, GC(1,2,4,8), GC(1,2,6,6), and GC(1,4,5,5) are all referred to as **UGC (unbalanced gray-code)**. In contrast, GC(3,4,4,4) is referred to as **BGC (balanced gray-code)** as reading different pages has roughly the same performance. Due to read retries at degraded reliability, overall read performance depends on the number of read retries as well, as shown in Equation 2.

TABLE III
COMPARISON AMONG VARIOUS GRAY-CODES.

Gray-code	TSP algorithm	Read.perf	Write.perf	Reliability
GC(1,2,4,8)	TSP(16,16), TSP(8,16), TSP(4,16)	low	high	low
GC(1,2,6,6)	TSP(16,16), TSP(4,16)	middle	high	low
GC(1,4,5,5)	TSP(16,16)	middle	low	high
GC(3,4,4,4)	TSP(16,16)	high	low	high

Table III compares different gray-codes, together with their supported TSPs, on read and write performance when using the best compatible TSP. The best TSP for each gray-code is shown in bold font in the table. Studies have shown that BGC is more robust in terms of reliability because the four bits of a QLC flash cell have similar reference voltages and it requires fewer read retries than that of UGC at degraded reliability.

III. MOTIVATION

In this section, we motivate our design by studying the performance and reliability characteristics of different TSPs and gray-codes. The industry has been focusing on improving the programming performance of TSPs. TSP(4,16) has the best write performance, but supports only UGC, i.e., GC(1,2,6,6), which leads to large latency and reliability variations across different pages in the same wordline. In particular, the LSB page has the smallest read latency and the best reliability

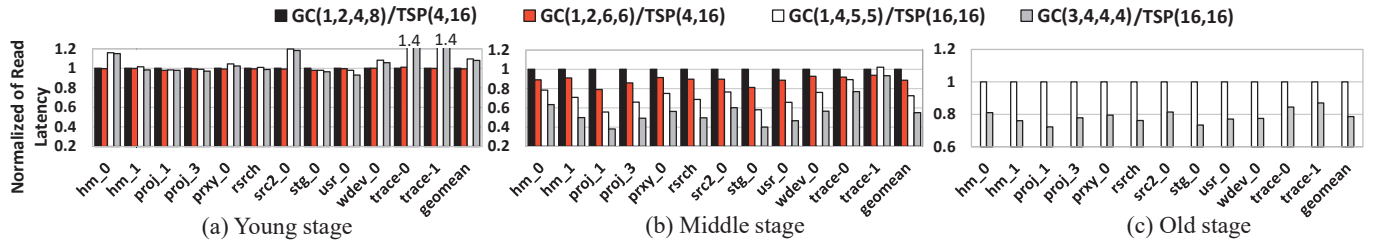


Fig. 3. Read performance comparison. (The figures are explained in Section III and related to Observation #1.)

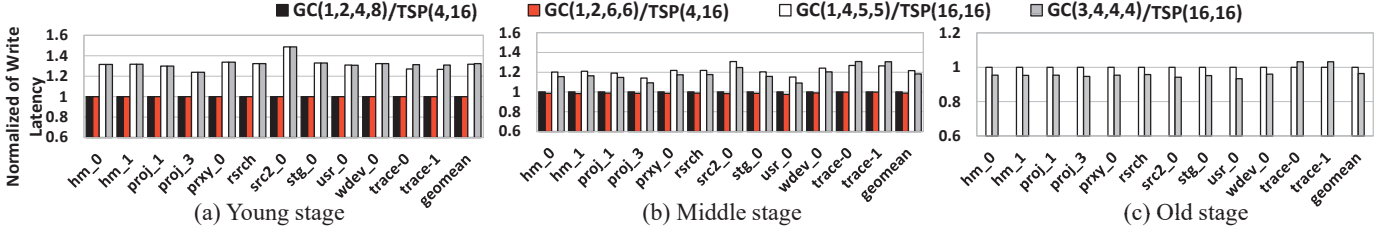


Fig. 4. Write performance comparison. (The figures are combined with Fig. 3 to get Observation #2.)

while the TSB page has the largest read latency and the worst reliability [53] — the latency gap between LSB and TSB pages can be as large as $20\times$ [56]. TSP(16,16) supports BGC but has the largest programming latency. To summarize, neither UGC nor BGC can achieve the best read and write performance, making it a dilemma to choose either one of them.

To study the impact of adopting different gray-codes in real applications, we conduct experiments on different applications at different stages. In this paper, we divide the reliability of flash into three stages (e.g., Young/Middle/Old) based on the number of retry operations with LDPC, as described below.

- **Young Stage:** this refers to the duration that the SSD just starts its service, which reflects the page reliability characteristics of different gray-codes when no retries occur.
- **Middle Stage:** this refers to the duration when the reliability of pages in UGC blocks is reduced due to the low reliability of MSB/TSB-based pages, while the reliability of pages in BGC blocks is still high, which requires few or no retries to perform a read operation. The real page reliability of different gray-codes is tested in experiments, which shows that MSB/TSB pages of UGC start retrying earlier than other pages (e.g., the TSB page of BGC). We then choose two sets of reliability features (termed Middle-1 and Middle-2 in Table IV) to reflect the reliability differences between gray-codes with decreasing reliability, and to facilitate further design of reliability-stage-aware gray-code arbitration.
- **Old Stage:** this is the duration when the number of retries exceeds the maximum value (e.g., 8 in this paper), the page reliability is further degraded, which cannot reliably store data and rewrite operations may be activated. We define the Old stage to discuss situations where a large number of retries occur on the MSB/TSB pages of UGC blocks.

Here, we present the rationale for defining three reliability stages, referring to existing work [56] and data collected from our real platform. First, retry operation occurs not only under the worst conditions but also under common cases in

modern high-density flash [56]. Second, we used an industrial-grade testing platform equipped with QLC chips to collect the retry number at 1K P/E cycles. The results verify that many retries occurred when reading pages, and MSB/TSB pages of UGC start retrying earlier than other pages of different gray codes. Third, it is common to protect data with more than 3-months retention time in high-density flash [30], especially for read-intensive scenarios, indicating that data reliability can be improved by selecting the high reliable gray-code. Based on the above understandings, we comprehensively consider the reliability characteristics of pages with different gray-codes to differentiate the specific stage and give the design principles. *Note that SSD duration is mainly at the Middle/Old stages. The duration of the Middle stage is the longest.*

The experimental settings are listed in Section VII-A. Due to the space limitation, we only use Middle-2 to represent the Middle stage in this paper. The performance results of Middle-1 can get the same observations and conclusions. Each gray-code adopts the fastest compatible TSP, as shown in Table III. Fig. 3-6 compare the read, write, and I/O performance with different gray-codes at three stages, respectively. For the Young/Middle stages, the results are normalized to GC(1,2,4,8) with TSP(4,16). From the results, the average read latency of GC(1,2,4,8) is around 390us at the Young stage and 1.3ms at the Middle stage, while the average write latency of GC(1,2,4,8) is around 15ms at the young stage and 18ms at the Middle stage. For the Old stage, the results are normalized to GC(1,4,5,5) with TSP(16,16). From the results, the average read latency of GC(1,4,5,5) is around 1.3ms, while the average write latency of GC(1,4,5,5) is around 24ms. Note that SSDs with GC(1,2,4,8) or GC(1,2,6,6) are not evaluated at the Old stage. This is because some pages of them are not reliable to store data based on the reliability stage definitions.

Observation #1: *The read latency of different gray-codes is similar at the Young stage, but presents significant variations with reliability degradation. At the Young stage (Fig. 3(a)), the exception is that the read performance of GC(3,4,4,4) and*

GC(1,4,5,5) may be worse than that of the other two gray-codes (e.g., *src2_0*, *trace-0*). The reason is that, although we use read request priority processing [50], the high programming latency from TSP(16,16) still introduces serious read and write conflicts, which degrade the read performance. At the Middle stage (Fig. 3(b)), the read latency in GC(3,4,4,4) is only 45% of that in GC(1,2,4,8). This comes from two reasons: (1) some pages (e.g., MSB/TSB pages) with UGC require more reference voltages than that of the other pages (e.g., LSB/CSB pages). (2) pages read from a larger number of reference voltages tend to be less reliable. As such, BGC provides better read performance for all stages, while UGC suffers from large latency spikes with reliability degradation. For example, the read latency distributions of workload *rsrch* are shown in Fig. 5. From the figure, the latency spikes, in particular, those from UGC, become more visible when comparing the latency statistics at the Middle stage over those at the Young stage.

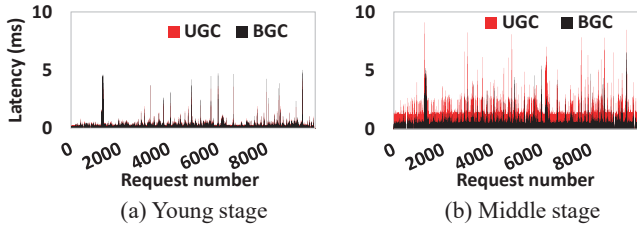


Fig. 5. Read latency spikes. (The x-axis represents requests with the time series, and the maximum value is 10,000.)

Observation #2: For all workloads, it is impossible for a single gray-code to achieve excellent read and write performance at the same time. First, GC(1,2,6,6) with TSP(4,16) achieves the best write performance (in Fig. 4(a) to 4(c)). However, with reliability degradation, GC(3,4,4,4) with TSP(16,16) has much better read performance than that of GC(1,2,6,6) with TSP(4,16) (in Fig. 3(a) to 3(c)). Second, GC(3,4,4,4) achieves the best read performance, but it is compatible with TSP(16,16) and thus leads to the worst write performance.

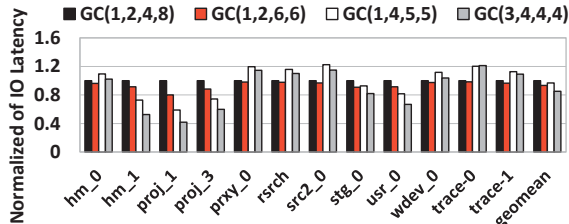


Fig. 6. I/O performance comparison. (The figure is related to Observation#3.)

Observation #3: Using a fixed gray-code cannot guarantee the I/O performance of workloads with different access patterns. Fig. 6 shows the overall I/O performance at the Middle stage. For read-dominated workloads, e.g., *hm_1*, using GC(3,4,4,4) can gain the best I/O performance. For write-dominated workloads, e.g., *prxy_0*, using GC(1,2,6,6) can gain the best I/O performance. Ideally, different gray-codes should be adopted for different workloads so as to achieve the best I/O performance to meet application-level requirements.

In this paper, we adopt GC(1,2,6,6) and GC(3,4,4,4) as two representative encoding for UGC and BGC, and apply TSP(4,16) and TSPs(16,16) for UGC and BGC, respectively.

IV. THE OVERVIEW OF MGC FLASH

In this section, we propose **MGC** to enable multiple gray-codes in the flash controller to exploit the advantages from both UGC and BGC to achieve the overall best I/O performance. Compared to the commercial SSD products that adopt a fixed gray-code based programming algorithm, MGC improves the read performance by exploiting the better reliability of BGC and reduces the programming latency by exploiting the high-speed programming of UGC. In addition, MGC helps to achieve better flash reliability and thus reduces the rewrites from internal read scrubbing or refresh mechanism [12] [41].

The proposed MGC consists of firmware and software modifications. At the firmware side, MGC enhances the programming process to support TSP(4,16) and TSP(16,16), and the read process to support data encoding/decoding using both UGC and BGC. The data decoding/encoding is to map 4-bit values from/to 16 voltage levels using two different mapping tables, which incurs negligible overhead. Supporting both TSP(4,16) and TSP(16,16) demands two programming sequences, which is hard to enable within one flash block. As such, to reduce implementation complexity, we enable TSP at block granularity, i.e., while both UGC and BGC are enabled, one flash block can only adopt one of them.

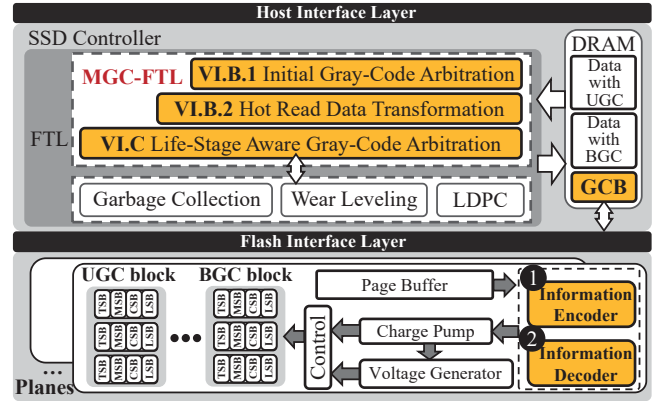


Fig. 7. Architecture of multiple-gray-code flash (§IV).

On the software side, we propose to make better use of MGC flash by matching the data access characteristics of workloads with the reliability characteristics of different gray-codes. Fig. 7 shows the architecture of MGC based SSDs. We propose a multiple-gray-code based FTL (MGC-FTL), which includes two techniques, a data access characteristics guided gray-code arbitration and a reliability-stage aware gray-code arbitration. Our major contribution of MGC-FTL is that data with different access characteristics are programmed with different TSPs and encoded with corresponding gray-codes. This enables us to achieve performance improvement for write-dominated data using UGC-based TSP and read-dominated data with BGC-based TSP. In addition, MGC-FTL exploits the

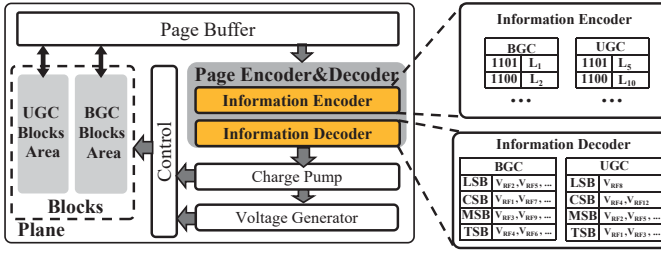


Fig. 8. Enabling MGC in Flash Controller (§V).

reliability characteristic for better gray-code selection. With reliability degradation, we gradually use the gray-code with high reliability, e.g., BGC, to replace the less reliable gray-code, e.g., UGC. In summary, we expect to achieve the overall best performance for the expected service lifetime of the SSDs.

V. FIRMWARE DESIGN FOR MGC FLASH

Flash programming is the process to charge flash cells on a WL to the voltage levels that match the input bit values from BLs. The matching is determined by a predefined encoding table between encoded bits and voltage levels. The programming process usually adopts an interactive program-verify strategy, which verifies the cell voltage after each device programming pulse and, if a cell reaches its desired voltage level, excludes it from the following programming pulses.

To enable flashing programming using two or more gray-codes, a flash controller needs to be enhanced to include two or more encoding tables (e.g., those in Table II), as shown in Fig. 8. For QLC, each encoding table has only 16 entries and thus the overhead for integrating multiple encoding tables is low. It is slightly more expensive to enable different TSPs, e.g., TSP(4,16) and TSP(16,16), in the system. While both are tested TSP programming strategies, TSP (4,16) programs cells to four coarse levels in the first step while TSP(16,16) to 16 levels. The latter demands more fine-granularity programming control. The controller needs to have the width and height of the programming pulses for each TSP.

Flash read also needs to be enhanced to adapt to multiple gray-codes. The controller exploits the encoding table of each gray-code to determine the number of reference voltage levels as well as the magnitude of each level.

To mitigate the hardware complexity, MGC is enabled at the block level, i.e., a block adopts the same gray-code and its TSP for all pages in this block. A block may adopt a different gray-code and its TSP once it is erased. Note that we track application usage characteristics at the page level, as we elaborate in the next section. Given there are two gray-codes adopted, we use a bitmap, referred to as gray-code bitmap (GCB), to track what gray-code is chosen for each block. The bitmap is small enough to be cached in DRAM, e.g., for a 1TB SSD with a 16MB block size, the bitmap is 8KB. We may need more bits per block if there are more than two gray-codes. MGC may also be enabled at different levels (e.g., plane and chip). We leave it to our future work.

Fig. 8 summarizes the firmware modification for MGC. We conducted a similar analysis of storage overhead as that in [38], one mapping table costs less than 0.07% of the flash area. In summary, the storage overhead is negligible.

VI. FTL FOR MGC FLASH MEMORY

A. Overview

A simple strategy for exploiting the benefits of a MGC design is to apply the fast programming algorithm, such as TSP(4,16), to the data with frequent writes, and the programming algorithm that supports the gray-code with superior read performance, such as TSP(16,16), for the data with frequent reads. In this way, it improves the access performance for both write-intensive and read-intensive data. However, we need to address the following challenges before its deployment.

- 1) We may not know the application-level access characteristics before execution. A sub-optimal gray-code selection could lead to low performance at runtime;
- 2) UGC, while exhibiting good read performance for LSB/CSB pages, has relatively low read performance for MSB/TSB pages. We need to decide how to split data among these pages to achieve overall performance improvement.
- 3) The read and write performance characteristics of UGC and BGC change with reliability degradation. A flash-reliability-oblivious arbitration may result in sub-optimal performance improvement at runtime.

To address these challenges, we propose MGC-FTL to exploit the MGC flash for performance and reliability optimization.

B. Access Characteristics Guided Gray-Code Arbitration

We first propose to exploit data access characteristics to make a better choice of gray-code at runtime. The I/O data can be classified into three types according to access characteristics: read-only, write-only, and interleave-accessing [35] [47] [33]. Prior studies from Li et al. [35] and Lv et al. [47] revealed that read-only data takes 85% of read requests, and write-only data takes 91% of write requests, on average. Only a small percentage of data is accessed in interleaving mode. Based on this observation, we adopt the following gray-code selection policy. If the data is identified as read-only, BGC should be adopted to optimize read performance. If the data is identified as write-only, UGC should be adopted to optimize write performance. Other data are determined based on access characteristics on-the-fly.

1) *Initial Gray-Code Arbitration:* To accommodate the fact that application-level data access characteristics are usually unknown prior to the execution, we propose a simple strategy that adopts BGC by default and switches to UGC if necessary at runtime. Such a decision comes from two folds: On the one hand, if the following accesses are mainly read operations, choosing read-performance-friendly BGC tends to provide good performance. If the following accesses are mainly write operations, due to the fact that flash writes are updated out-of-the-place, we can choose a different gray-code (i.e., UGC) at write time. The performance penalty to such type of data is only the first write, which is small in general. On the other

hand, by considering the interaction with TSP, TSP(4,16) and TSP(16,16) can start cell programming after buffering two and four pages, respectively. That is, the slow TSP(16,16) tends to introduce a larger queuing delay.

2) *Hot Read Data Transformation*: When integrating different gray-codes in the SSD, there exist three types of flash pages based on their asymmetric read latency:

- *Type-1 page*: this refers to the TSB or MSB pages in UGC blocks. They tend to have high read latency.
- *Type-2 page*: this refers to all the pages in BGC blocks. Their read latency is in the middle range.
- *Type-3 page*: this refers to the LSB or CSB pages in UGC blocks. Their read latency is relatively low.

Given read operations do not update the data, if there is a mismatch between the hotness of the read-intensive data and the location of the data, it might be beneficial to transform the encoding for such data. For example, if data are written in UGC, the read performance for LSB/CSB pages is superior. However, at runtime, we may accumulate more reads to the MSB/TSB pages such that it is beneficial to transform the encoding to BGC. In this work, read hotness is used as the indicator to guide read data transformation while other indicators also can be applied, such as the performance requirement hint from the host system [41]. If their read hotness continue to increase, these MSB/TSB pages (now in the BGC block) may be moved to LSB/CSB pages in the UGC block.

To implement it, we evaluate the tradeoff between transformation overhead and benefit to decide the appropriate read hotness threshold. The evaluation is based on a simple heuristic that a page, once read s times, is likely to be read again at least s times. If the transformation overhead, i.e., writing QLC with P/E cycle reduction and write latency, is less than the benefits from reading faster pages s times, we may trigger such a transformation.

To differentiate read hotness, we set two thresholds, i.e., T_{UR} and T_{BR} , as the read (or read-retry in LDPC) numbers on MSB/TSB pages in UGC blocks (Type-1 pages) and BGC blocks (Type-2 pages), respectively. We collect page-level hotness statistics in the SSD controller. Since the read latency gap between Type-1 pages and Type-2 pages is smaller than that between Type-2 pages and Type-3 pages, T_{UR} is set more aggressively than T_{BR} , which tends to have good read performance improvement. In this paper, T_{UR} is set to 1 while T_{BR} is studied together with the transformation overhead.

3) *Garbage Collection and Wear Leveling*: One critical component inside SSD is garbage collection, which reclaims invalid pages caused by data updates [69]. For the above gray-code arbitration scheme, UGC blocks are prone to garbage collection since there are more invalid pages. During garbage collection, valid data should be collected and programmed to UGC blocks. This is because garbage collection is time-consuming, and adopting fast programming with UGC can speed up the process. If a BGC block is selected for garbage collection, the valid data should still be stored in the BGC block due to that data in BGC blocks are read-dominant. Actually, BGC blocks are less likely to be garbage collected as

they frequently store read-intensive data. Therefore, traditional garbage collection can be adopted with little modifications.

Another critical component is wear leveling that even the wearing among blocks for lifetime optimization [68] [21] [42]. For the above gray-code scheme, UGC blocks are erased more frequently than that of BGC blocks. In this case, we propose to encode blocks with UGC or BGC in an interleaving way. For example, if a UGC block is erased, it will be used as a BGC block next time during dynamic wear leveling. However, since BGC blocks are rarely erased, we need to select the BGC blocks with the lowest erase counts during static wear leveling.

C. Reliability-Stage Aware Gray-Code Arbitration

As presented in Motivation, UGC is sensitive to reliability degradation while BGC is not. In this part, by exploiting the characteristics at three reliability stages as discussed in Section III, MGC further optimizes the gray-code arbitration as follows. We use an example in Fig. 9 to facilitate the discussion. For each stage, we discuss the gray-code arbitration on three operations: (1) *reads*; (2) *write for the first time*; (3) *updates*.

1) *Young Stage Gray-Code Arbitration*: For the Young stage, pages programmed with TSP(4,16) and encoded with UGC have superior write performance and comparable read performance to that of BGC. Then, we propose to use UGC as the default gray-code. In this case, the read and write performance are both excellent. More importantly, there is no gray-code transformation cost. As shown in Fig. 9(a), all data are encoded with UGC and programmed with TSP(4,16). That is, there are only UGC blocks in flash memory.

2) *Middle Stage Gray-Code Arbitration*: For the Middle stage, data programmed with TSP(4,16) and encoded with UGC may introduce retries on the TSB and MSB based pages. Two changes are made compared with the Young stage. First, BGC is chosen as the initial gray-code to encode data for the first writes. Second, the data on MSB/TSB pages in UGC blocks are transformed to BGC blocks when the read counts exceed T_{UR} . As shown in Fig. 9(b), data A-H are written to BGC blocks for the first time (Step ①), and then updated to UGC blocks (Steps ③ and ④). Then, data A and C in the BGC block are frequently accessed and the read count exceeds the threshold, but the transformation is not triggered since the reliability of BGC blocks is still high (Step ②). However, if the read count of the data D and E stored in the UGC blocks exceeds the threshold, they will be transferred to BGC blocks for read performance optimization (Step ⑤).

3) *Old Stage Gray-Code Arbitration*: For the Old stage, data encoded with UGC using TSP(4,16) are no longer reliable. In this case, UGC blocks cannot work and their lifetime is stopped at the Middle stage. BGC will still be selected as the initial gray-code for reliability concerns. Since the LSB/CSB pages of UGC blocks are still reliable, we transfer the hot read data to these two pages when they are identified in the BGC blocks. However, due to the low reliability of UGC, only the first step of TSP(4,16) is processed to program the two pages in one WL. For the data previously programmed to UGC blocks, the data at the MSB/TSB pages should be

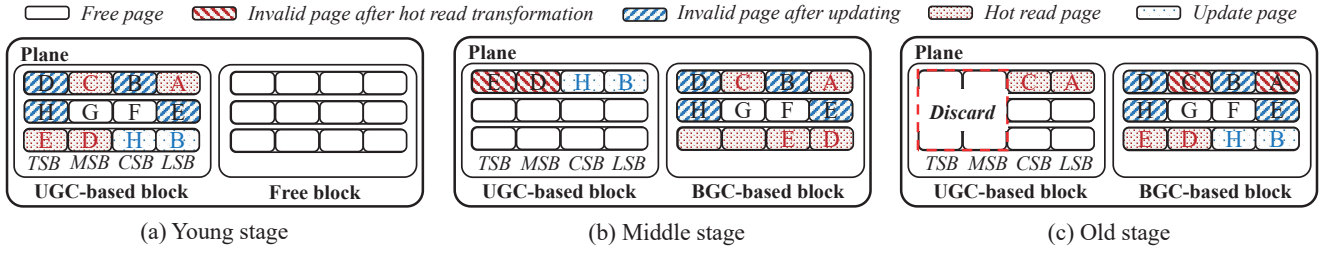


Fig. 9. Reliability-stage aware gray-code arbitration for the following operation steps: ❶ Data A-H are written for the first time; ❷ Data A and C are frequently read; ❸ Data B and H are updated; ❹ Data D and E are updated; ❺ Data D and E are frequently read. (As described in Section VI-C.)

transferred to BGC blocks during rewrites. As shown in Fig. 9(c), data A-H are written to BGC blocks for the first time (Step ❶). For data updates, they will be written to BGC blocks (Steps ❸ and ❹). If frequently accessed data are from BGC blocks, they will be transferred to the LSB or MSB pages of UGC blocks for read performance optimization (Step ❷).

VII. EVALUATION

A. Experiment Setup

1) *SSD Configuration*: Considering no real SSDs supporting multiple-gray-codes, simulation-based schemes are used to evaluate the benefits of MGC-FTL. First, a widely used simulator, SSDsim [19] is employed with extensions to support reliability modeling and features of 3D QLC flash. Second, most parameters are based on recent works [56] [26] and confirmed with our industry partners, extending them to 3D QLC flash. The configurations for the simulated SSD are listed in Table IV(a). It has four channels, with two chips per channel and four planes per chip. Each chip has 2048 blocks, and each block has 1024 16KB pages. In the controller, the default page mapping and the existing bad block management scheme are adopted. t_R is the latency to read a page from flash, which depends on the page type with different numbers of sensing voltages according to Equation 1. t_{SENSE} is set to 22.67 μ s, which is the latency of applying one sensing voltage. t_{PROG} is the latency for programming a page, which is different among different TSPs. t_{DMA} is set to 13 μ s with 1200MT/s I/O rate [26], and t_{LDPC} is set to 20 μ s [56]. The adopted LDPC can correct 72-bit errors per 1KiB code-word [56].

2) *Reliability Modeling*: In this section, we elaborate the reliability model adopted in this paper. Table IV(b) summarizes the read latency results at three reliability stages (as defined in Section III and VI-C). We exploited an industrial-grade testing platform from YEESTOR and validated the results against QLC flash chips from main flash manufacturers.

Retry Modeling: According to prior work [27] [56] and open reliability model [71], we can develop the retry model like Equation 3 to determine the average number of retry operations (n_{RETRY}) for each page in a specific stage.

First, to show the RBER (raw bit error rate) of the data page, we defined a variable, α , which reflects the RBER for two adjacent voltage states ($RBER_{SENSE}$). Then, the RBER of a data page with the number of reference voltages (n_{SENSE}) can be expressed as $\alpha * RBER_{SENSE} * n_{SENSE}$, which should be collected from the real platform.

Second, to show the capability of the equipped LDPC, we use E_{LDPC} to represent the maximum number of error bits in a data page that the LDPC can correct. If the RBER of the currently accessed page is less than E_{LDPC} , the accessed data can be correctly read. Otherwise, retry operations occur until the data is read correctly or the number of retries is exhausted.

Finally, to model the retry process, we use a variable δ to represent the RBER reduction for each read retry. δ is highly related to the retry algorithm (such as a predefined retry voltage table), which varies by flash manufacturers or products with different requirements. Previously, Kim et al. [27] model threshold voltage tuning and soft-decision decoding, effectively reducing the RBER for each read by a retry scale factor. The retry scale factor can determine δ , e.g., setting the retry scale factor to 2 means that each retry can reduce the RBER by 50% for MLC/TLC flash [27]. To evaluate QLC flash, we set a more conservative retry scale factor to 5, which can reduce 20% of RBER for each retry. Then, δ is set to 20%, that is, after each retry, the RBER of the read page can be reduced to 80% of its original RBER. Based on the above definitions, the retry modeling can theoretically be constructed as Equation 3. After calculation, the minimum retry number can be expressed by Equation 4. Note, if n_{RETRY} is higher than the maximum retry number, the retry process is terminated, and the soft decision of the LDPC will be activated to recover the data.

$$(\alpha * RBER_{SENSE} * n_{SENSE}) * (1 - \delta)^{n_{RETRY}} \leq E_{LDPC} \quad (3)$$

$$n_{RETRY} \geq \log_{(1-\delta)} \left(\frac{E_{LDPC}}{\alpha * RBER_{SENSE} * n_{SENSE}} \right) \quad (4)$$

Retry Modeling for Different Reliability Stages: To determine the number of page retries for the three stages, the RBER of pages under the four gray-codes is collected with real QLC flash chips. We use a YEESTOR platform, which is equipped with 176L QLC flash chips. The tested flash chips are erased by 1K P/E cycles, and random data are programmed. To acquire the RBER of data with retention time at room temperature, the flash chips were heated at a high temperature for several hours of baking time to accelerate the retention loss, which refers to Arrhenius' Law. [45].

Based on the collected RBER, we can calculate the number of retries according to the retry modeling (Equation 4). Fig. 10 shows the characteristics of retry operations. Darker lines represent pages with higher n_{SENSE} . From the results, the number of retries can be divided into five cases: ❶ When n_{SENSE} is 8 (e.g., TSB page of GC(1,2,4,8)), the

TABLE IV
THE CHARACTERISTICS OF SIMULATED SSD AND WORKLOADS.

(a) SSD parameters		(b) Read latency (in <i>us</i>) and retry number					(c) Characteristics of evaluated workloads															
(channel,chip)	(4,2)	nSENSE	Young	Middle-1	Middle-2	Old	Trace	RF (GB)	WF (GB)	RS (KB)	WS (KB)	R (GB)	W (GB)	RR (%)	Trace	RF (GB)	WF (GB)	RS (KB)	WS (KB)	R (GB)	W (GB)	RR (%)
(plane,block,page)	(4,2048,1024)	1	56 (0)	56 (0)	56 (0)	112 (1)	<i>hm_1</i>	0.14	0.10	18	23	8.54	0.35	96.1	<i>wdev_0</i>	0.20	0.30	17	12	3.18	9.23	25.6
page size	16KB	2	78 (0)	78 (0)	78 (0)	234 (2)	<i>proj_1</i>	26.2	1.39	43	22	30.6	1.98	95.0	<i>hm_0</i>	0.65	0.67	12	11	2.76	8.03	25.6
cell density	QLC	3	101 (0)	101 (0)	202 (1)	505 (4)	<i>proj_3</i>	2.72	1.58	15	30	13.0	2.78	82.3	<i>src2_0</i>	0.40	0.41	13	11	1.63	9.09	15.2
average tR [26]	85 <i>us</i>	4	124 (0)	124 (0)	372 (2)	868 (6)	<i>usr_0</i>	2.06	0.34	47	14	16.8	8.12	67.4	<i>rsrch</i>	0.07	0.24	16	13	1.35	11.0	11.0
tPROG(16,16) [20]	2150 <i>us</i>	5	146 (0)	146 (0)	730 (4)	1460 (9)	<i>stg_0</i>	6.16	0.23	34	13	7.39	9.31	44.3	<i>prxy_0</i>	0.08	0.11	10	6	0.27	5.81	4.50
tPROG(8,16) [25]	2080 <i>us</i>	6	169 (0)	338 (1)	1183 (6)	N/A	<i>trace-0</i>	1.14	4.8	25	836	2.09	9.75	17.7	<i>trace-1</i>	0.62	0.37	10	750	0.99	0.74	57.3
tPROG(4,16) [26]	1630 <i>us</i>	8	214 (0)	642 (2)	2140 (9)	N/A																

number of retries is 7 to 11. ② When $nSENSE$ is 5 or 6 (e.g., MSB/TSB pages of GC(1,2,6,6) and GC(1,4,5,5)), the retries reach 5 to 10. ③ When $nSENSE$ is 4 (e.g., CSB/MSB/TSB pages of GC(3,4,4,4)), the number of retries is 3 to 9. ④ When $nSENSE$ is 2 or 3 (e.g., CSB page of GC(1,2,4,8), LSB page of GC(3,4,4,4)), the number of retries is 1 to 5. ⑤ When $nSENSE$ is 1 (e.g., LSB page of GC(1,2,4,8)/GC(1,2,6,6)/GC(1,4,5,5)), no retries occur.

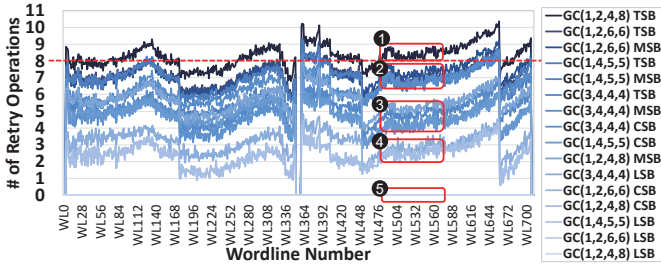


Fig. 10. Characteristics of Retry Operations.

Then, we can count the retry values (in Table IV(b)) for pages with different $nSENSE$ under the three reliability stages. The number of retries increases from the Young to the Old stages, and the retries are larger with a higher $nSENSE$. First, the pages of all gray-codes are highly reliable and no retries occur when reading (referring to the Young stage). Second, as the reliability degradation, the MSB/TSB pages with UGC start a retry operation when reading. The number of retry values of these pages ($nSENSE=6$) is set to 1 (referring to the Middle-1 stage). Therefore, pages with lower $nSENSE$ have a retry value of 0. Third, as reliability drops further, the LSB pages in BGC blocks start retrying, set to 1 (referring to the Middle-2 stage). Forth, the LSB pages of UGC blocks start retrying, set to 1 (referring to the Old stage). There is a large number of retries on MSB/TSB pages in UGC blocks and rewrite operations are triggered, similar to the Old stage, as shown in Figure 10. Then, at the Old stage, the retry values of MSB/TSB pages in UGC blocks are set to a value greater than the upper limit of retry operations (e.g., 8 in this paper), such as 9. For other pages with different values of $nSENSE$, the average number of retries is set based on the above-collected RBER trend and retry modeling (e.g., we select 4 and 6 for the LSB page of BGC and CSB/MSB/TSB pages of BGC at the Old stage, respectively). Overall, the retry values in Table IV(b) are empirically set with reliability trends tested in real chips and stages defined in this paper to understand

the page reliability characteristics of different gray-codes. In practice, the number of retries is complicated and varies for chips from different flash manufacturers or to account for process variations. Due to the limited scope of discussion in this paper, this part will serve as our future work.

Furthermore, considering real SSD scenarios, the number of retries tends to have a Gaussian-like distribution [61] [5]. That is, for the average retry values given in Table IV(b), we use a Gaussian distribution, $\mathcal{N}(\mu, \sigma^2)$, to determine the actual retries of the accessed page to accurately simulate real scenarios. μ is set based on the average number of retries given in Table IV(b), and σ is set to 1 for simplicity.

3) *Evaluated Workload*: Ten workloads from MSR [49] and two from YCSB Workload on RocksDB [66] [65] were used, as listed in Table IV(c), including read/write footprint (RF/WF), read/write request size (RS/WS), total read/write volume (R/W), and read ratio (RR). The footprint represents the size of the space for data access. Note that these workloads are updated and widely used to evaluate the state-of-the-art SSDs [66] [31].

B. Experimental Results

To fully demonstrate the benefit of MGC-FTL, we evaluated five schemes, including UGC, UGC_RLV, BGC, MGC and MGC_Stage. UGC [26] and BGC [32] represent the state-of-the-art single-gray-code based flash. UGC_RLV is a read latency variation aware scheme with UGC-based QLC flash [47], which exploits the LSB and CSB pages of UGC blocks for read performance optimization. MGC represents the proposed scheme with the access characteristics guided gray-code arbitration. MGC_Stage is the enhanced MGC with the reliability-stage aware gray-code arbitration.

In the following experiments, we evaluated the performance, tail latency, flash lifetime and sensitivity studies. We have done the above experiments at all stages, and the results meet our expectations. However, the duration of the SSD is mainly at the Middle/Old stages (as described in Section III) and space limitation, only the results at the Middle/Old stages are discussed. Additionally, in the Old stage, UGC blocks cannot work due to the low reliability of MSB/TSB pages, thus UGC and MGC are not evaluated. Then, the results are normalized to UGC at the Middle stage and BGC at the Old stage. Here, we use the parameters of Middle-2 to evaluate the Middle stage since both Middle-1 and Middle-2 can reflect the worse reliability of UGC as the definition of the Middle stage.

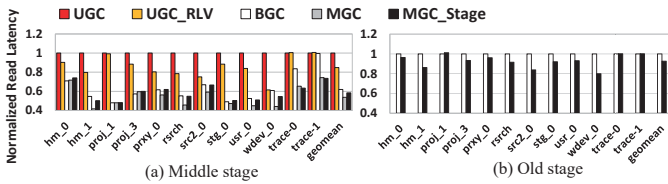


Fig. 11. Read performance comparison.

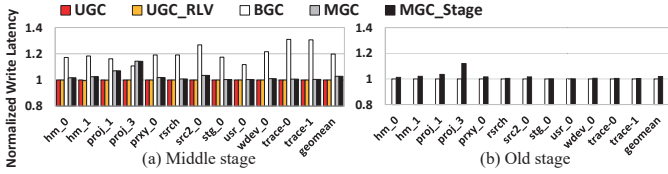


Fig. 12. Write performance comparison.

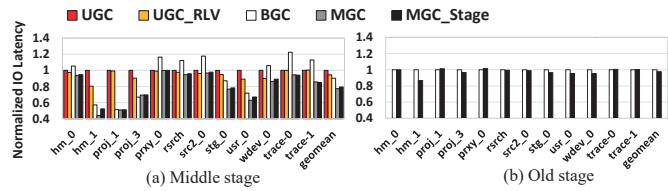


Fig. 13. I/O performance comparison.

Read performance: Fig. 11 shows the normalized read latency. In the Middle stage, the read performance of BGC is much better than that of UGC for most workloads. This is because MSB/TSB pages in UGC blocks suffer from reliability/performance degradation. By exploiting the read latency variation among pages in UGC blocks, UGC_RLV reduces the latency by 15% compared with UGC. MGC that arbitrates BGC for the coming data can achieve similar read performance to that of BGC. Furthermore, the read latency of MGC_Stage is slightly higher than that of MGC by 4.8% due to that several frequently read data in BGC blocks are not transferred to LSB or CSB pages of UGC blocks for overhead consideration. In the Old stage, compared with BGC, the read performance of MGC_Stage can be optimized by 7.4% on average. This is because unreliable UGC blocks cannot store data, which are discarded at the Old stage. To sum up, compared with UGC_RLV, MGC_Stage improves the read performance by 26% on average and up to 51% at the Middle stage. At the Old stage, the read performance of MGC_Stage can be improved by 7.4% on average compared with BGC.

Write performance: Write performance is highly related to the adopted TSPs, as shown in Fig. 12. In the Middle stage, BGC/TSP(16,16) introduces 12%-31% higher write latency compared with UGC/TSP(4,16). Such a write performance gap is smaller than the theoretical value due to that the read latency of UGC blocks increases significantly, which will affect the write performance of SSDs due to the read priority policy. Differently, the write performance of MGC and MGC_Stage is similar to that of UGC, as we only use BGC for the first write and arbitrate UGC for updates. In the Old stage, BGC and MGC_Stage have similar write latency, as we arbitrate

BGC to encode data for the first write and further updates.

I/O performance: Fig. 13 presents the normalized I/O latency, which highly relates to the read/write ratio of the workload (as shown in Table IV(c)). I/O performance depends primarily on read performance optimization for read-intensive workloads (e.g., *hm_1*) and write performance optimization for write-intensive workloads (e.g., *prxy_0*). MGC and MGC_Stage can achieve good I/O performance improvement no matter whether the workload is read-intensive or write-intensive.

Tail latency: To further understand the latency stability, we collected the tail latency at the 99th percentile of the Middle stage, as shown in Fig. 14. First, BGC can reduce the tail latency by 42% on average compared with that of UGC. Second, UGC_RLV only focuses on average performance optimization, where the tail latency is reduced by 12%, on average, over UGC. Some workloads experienced no improvement in tail latency. Third, the tail latency of both MGC and MGC_Stage is similar to that of BGC due to that frequently read data are stored in BGC blocks with high reliability, which can achieve stable performance. In conclusion, MGC_Stage can reduce the read latency by 39% at two nines compared with UGC.

Lifetime: We then evaluate the factors that impact flash lifetime by collecting the rewrite counts and transformation cost (e.g., read and rewrite operations during gray-codes conversion). First, rewrites are triggered to protect data reliability. If the number of retries during data accessing exceeds a limit, the data should be read and rewritten to other places. If flash reliability is low, too many rewrites may reduce lifetime and even make SSD fail. Second, during gray-code conversion, some pages will be read out and programmed to other WLS.

Fig. 15 shows the write amplification factor (WAF), where a smaller value of WAF means less impact on flash lifetime. From the results, BGC introduces much less WAF compared with UGC. More importantly, compared with existing works and MGC, MGC_Stage only produces a little lifetime reduction for three reasons. First, MGC_Stage can improve data reliability, which will reduce the rewrites. Second, we adopt a conservative hot read data transformation method to minimize transformation costs. Third, we tend to use BGC at the Old stage to tame the low reliability of UGC, which can make the SSD still work at the Old stage. Also, we collected the capacity reduction at the Old stage, which is represented by the number of UGC blocks used, as shown in Fig. 16. The results show that the reduced capacity is around 0.6%-7.8% and 1.7% on average due to that we only used a small portion of blocks as UGC-based blocks to optimize the read performance. In conclusion, MGC_Stage achieves a similar lifetime as BGC.

Sensitivity study: One critical parameter is the transformation threshold T_{BR} . Fig. 17 shows the normalized read latency and transformation cost for workload *wdev_0* at the Middle stage by varying T_{BR} from 2 to 8. A smaller T_{BR} triggers more aggressive migrations. Then, more data are speeded by transferring them to the pages with lower latency. That is, a better read performance can be achieved with a higher migration cost. From the results, the overall migration cost is

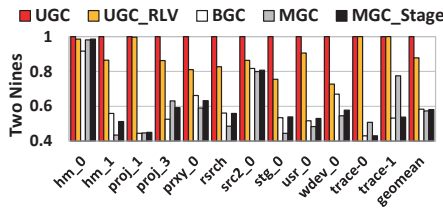


Fig. 14. Tail latency comparison.

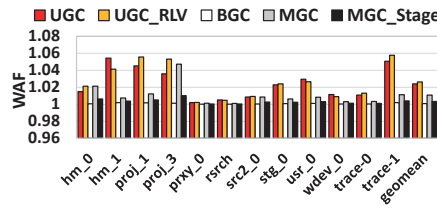


Fig. 15. Lifetime reduction.

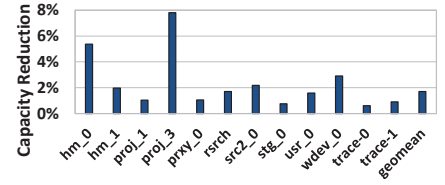


Fig. 16. Capacity reduction.

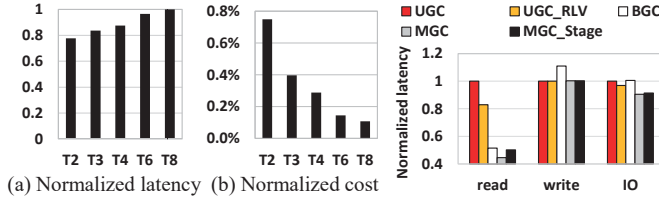


Fig. 17. Sensitivity study of T_{BR} .

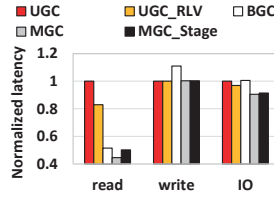


Fig. 18. Garbage collection.

small, i.e., about 0.1%-0.8% for the evaluated T_{BR} . To obtain a better trade-off between read performance and migration cost, T_{BR} is set to 3 in this paper. In the future, we can apply the hot data identification using bloom filters or using the machine learning method [54] [2] to MGC.

Garbage collection: To evaluate the performance of our proposed method under garbage collection, we collected the average read/write/IO latency at the Middle stage. The results of *trace-0* are used as an example in Fig. 18. The results show that MGC_Stage is not affected by garbage collection, and the conclusion is the same as the case without garbage collection.

VIII. RELATED WORK

RBER reduction: The basic idea is to reduce the raw bit error rate (RBER) to enhance flash reliability and lifetime. For example, many studies proposed to reduce RBER, such as the adoption of fine-grained and multi-step programming algorithms [35] [32] [20] [60] [26]. Some prior works proposed applying refresh schemes to control the retention errors, which can obtain higher P/E cycles [10] [44] [15] [51]. Some other works presented to adjust the read reference voltages to reduce the RBER and achieve high read performance [52] [58] [7] [8]. All techniques mentioned above attempt to tame the challenge of RBER reduction. Different from prior work, we proposed to improve the performance of high-density flash memory by exploring the advantages of different gray-codes.

LDPC optimization: Wu et al. [64] proposed to exploit the write interference and retention time to optimize the decoding process of LDPC. Du et al. [16] proposed to carefully select the sensing voltage for LDPC. Li et al. [36] proposed to smartly place the sensing voltage of LDPC. The above works reduce the decoding latency of LDPC by improving the quality of the input information. However, none of them consider the reliability characteristics of gray-codes.

Latency variation optimization: Prior works proposed to explore the latency variation among pages to improve the read

performance. For example, Cui et al. [13] [14] proposed to schedule I/O requests based on the latency variation to improve read performance. Li et al. [34] proposed to exploit the process variation to improve read performance. Lv et al. [47] proposed a latency variation aware data placement scheme to explore the latency difference among pages in a QLC flash to maximize read performance. Our work has further explored the latency variation among different gray-codes and applied the MGC scheme to different reliability stages for lifetime consideration.

Coding optimization: Prior works proposed to optimize the coding of high-density flash memory, such as WOM codes [23] [67] [48]. For example, Jaffer et al. proposed to use WOM codes to gain more program count before the block is erased, which can enhance the flash lifetime [23]. For gray-codes, there are also some works that analyze the reliability among different pages. For example, Pletka et al. [59] and lv et al. [47] proposed read performance optimization schemes based on asymmetric gray-codes. Liu et al. [43] studied the QLC program and read mechanisms, and then discussed five program algorithms and four gray-codes to match different program algorithms. Different from the above works, our proposal is the first work on designing multiple-gray-codes for 3D NAND flash-based high-density SSDs to meet the dynamic read and program performance requirements at both application and device levels.

IX. CONCLUSION

In this paper, we proposed and evaluated MGC for high-density 3D NAND flash memory. MGC enhances the flash controller to enable the selection of appropriate gray-code at runtime, which helps to match the performance characteristics at application and flash levels. Our experimental evaluations show that MGC achieves better performance and lifetime guarantee compared with the state-of-the-arts.

ACKNOWLEDGMENTS

We would like to thank anonymous reviewers for their valuable comments. This work was supported by the Excellent PhD Student Promotion Project of East China Normal University (YBNLTS2021-036), the NSFC 62072177, 62202396 and 62102219, and the Shanghai Science and Technology Project (20ZR1417200, 22QA1403300). The corresponding author is Liang Shi (lshi@cs.ecnu.edu.cn).

REFERENCES

- [1] Y. Aiba, H. Tanaka, T. Maeda, K. Sawa, F. Kikushima, M. Miura, T. Fujisawa, M. Matsuo, and T. Sanuki, "Cryogenic operation of 3D flash memory for new applications and bit cost scaling with 6-bit per cell (HLC) and beyond," in *IEEE Electron Devices Technology Manufacturing Conference (EDTM)*, 2021.
- [2] J. Bhimani, N. Mi, and B. Sheng, "Bloomstream: Data temperature identification for flash based memory storage using bloom filters," in *IEEE International Conference on Cloud Computing (CLOUD)*, 2018.
- [3] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Errors in flash-memory-based solid-state drives: Analysis, mitigation, and recovery," *CoRR*, vol. abs/1711.11427, 2017.
- [4] Y. Cai, S. Ghose, Y. Luo, K. Mai, O. Mutlu, and E. F. Haratsch, "Vulnerabilities in MLC NAND flash memory programming: Experimental analysis, exploits, and mitigation techniques," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017.
- [5] Y. Cai, E. F. Haratsch, O. Mutlu, and K. Mai, "Threshold voltage distribution in MLC NAND flash memory: Characterization, analysis, and modeling," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2013.
- [6] Y. Cai, Y. Luo, S. Ghose, and O. Mutlu, "Read disturb errors in MLC NAND flash memory: Characterization, mitigation, and recovery," in *Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2015.
- [7] Y. Cai, Y. Luo, E. F. Haratsch, K. Mai, and O. Mutlu, "Data retention in MLC NAND flash memory: Characterization, optimization, and recovery," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2015.
- [8] Y. Cai, O. Mutlu, E. F. Haratsch, and K. Mai, "Program interference in MLC NAND flash memory: Characterization, modeling, and mitigation," in *IEEE International Conference on Computer Design (ICCD)*, 2013.
- [9] Y. Cai, G. Yalcin, O. Mutlu, E. F. Haratsch, A. Crista, O. S. Unsal, and K. Mai, "Error analysis and retention-aware error management for nand flash memory," *Intel Technology Journal*, vol. 17, no. 1, 2013.
- [10] Y. Cai, G. Yalcin, O. Mutlu, E. F. Haratsch, A. Crista, O. S. Unsal, and K. Mai, "Flash correct-and-refresh: Retention-aware error management for increased flash memory lifetime," in *IEEE International Conference on Computer Design (ICCD)*, 2012.
- [11] W. Cho, J. Jung, J. Kim, J. Ham, S. Lee, Y. Noh, D. Kim, W. Lee, K. Cho, K. Kim, H. Lee, S. Chai, E. Jo, H. Cho, J.-S. Kim, C. Kwon, C. Park, H. Nam, H. Won, T. Kim, K. Park, S. Oh, J. Ban, J. Park, J. Shin, T. Shin, J. Jang, J. Mun, J. Choi, H. Choi, S.-W. Choi, W. Park, D. Yoon, M. Kim, J. Lim, C. An, H. Shirr, H. Oh, H. Park, S. Shim, H. Huh, H. Choi, S. Lee, J. Sim, K. Gwon, J. Kim, W. Jeong, J. Choi, and K.-W. Jin, "A 1-Tb, 4b/Cell, 176-Stacked-WL 3D-NAND Flash Memory with Improved Read Latency and a 14.8Gb/mm² Density," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [12] J. Cui, W. Liu, J. Huang, and L. T. Yang, "Exploiting disturbance-aware read redirection for performance improvement in 3D flash memory," in *ACM Great Lakes Symposium on VLSI (GLSVLSI)*, 2020.
- [13] J. Cui, W. Wu, X. Zhang, J. Huang, and Y. Wang, "Exploiting latency variation for access conflict reduction of NAND flash memory," in *IEEE Symposium on Mass Storage Systems and Technologies (MSST)*, 2016.
- [14] J. Cui, Y. Zhang, W. Wu, J. Yang, Y. Wang, and J. Huang, "Div: Exploiting device level latency variations for performance improvement on flash memory storage systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 37, no. 8, 2017.
- [15] Y. Du, Q. Li, L. Shi, D. Zou, H. Jin, and C. J. Xue, "Reducing LDPC soft sensing latency by lightweight data refresh for flash read performance improvement," in *ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017.
- [16] Y. Du, D. Zou, Q. Li, L. Shi, H. Jin, and C. J. Xue, "Laldpc: Latency-aware LDPC for read performance improvement of solid state drives," in *IEEE Symposium on Mass Storage Systems and Technologies (MSST)*, 2017.
- [17] F. Gray, "Pulse code communication," *United States Patent Number 2632058*, 1953.
- [18] T. Higuchi, T. Kodama, K. Kato, R. Fukuda, N. Tokiwa, M. Abe, T. Takagiwa, Y. Shimizu, J. Musha, K. Sakurai, J. Sato, T. Utsumi, K. Yoneya, Y. Suematsu, T. Hashimoto, T. Hioka, K. Yanagidaira, M. Kojima, J. Matsuno, K. Shiraishi, K. Yamamoto, S. Hayashi, T. Hashiguchi, K. Inuzuka, A. Sugahara, M. Honma, K. Tsunoda, K. Yamamoto, T. Sugimoto, T. Fujimura, M. Kaneko, H. Date, O. Kobayashi, T. Minamoto, R. Tachibana, I. Yamaguchi, J. Lee, V. Ramachandra, S. Rajendra, T. Tang, S. Darne, J. Lee, J. Li, T. Miwa, R. Yamashita, H. Sugawara, N. Ookuma, M. Kano, H. Mizukoshi, Y. Kuniyoshi, M. Watanabe, K. Akiyama, H. Mori, A. Arimizu, Y. Katano, M. Ehama, H. Maejima, K. Hosono, and M. Yoshihara, "A 1Tb 3b/Cell 3D-Flash Memory in a 170+ Word-Line-Layer Technology," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021.
- [19] Y. Hu, H. Jiang, D. Feng, L. Tian, H. Luo, and C. Ren, "Exploring and exploiting the multilevel parallelism inside SSDs for improved performance and endurance," *IEEE Transactions on Computers (TC)*, vol. 62, no. 6, 2012.
- [20] H. Huh, W. Cho, J. Lee, Y. Noh, Y. Park, S. Ok, J. Kim, K. Cho, H. Lee, G. Kim, K. Park, K. Kim, H. Lee, S. Chai, C. Kwon, H. Cho, C. Jeong, Y. Yang, J. Goo, J. Park, J. Lee, H. Kiri, K. Jo, C. Park, H. Nam, H. Song, S. Lee, W. Jeong, K.-O. Ahn, and T.-S. Jung, "A 1Tb 4b/cell 96-Stacked-WL 3D NAND flash memory with 30MB/s program throughput using peripheral circuit under memory cell array technique," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020.
- [21] S. Im and D. Shin, "Differentiated space allocation for wear leveling on phase-change memory-based storage device," *IEEE Transactions on Consumer Electronics*, vol. 60, no. 1, 2014.
- [22] K. Ishimaru, "Future of non-volatile memory from storage to computing," in *Annual IEEE International Electron Devices Meeting (IEDM)*, 2019.
- [23] S. Jaffer, K. Mahdavi, and B. Schroeder, "Improving the reliability of next generation SSDs using WOM-v codes," in *USENIX Conference on File and Storage Technologies (FAST)*, 2022.
- [24] T. Jiang, G. Zhang, Z. Huang, X. Ma, J. Wei, Z. Li, and W. Zheng, "FusionRAID: Achieving consistent low latency for commodity SSD arrays," in *USENIX Conference on File and Storage Technologies (FAST)*, 2021.
- [25] P. Kalavade, "4 bits/cell 96 layer floating gate 3d nand with cmos under array technology and ssds," in *IEEE International Memory Workshop (IMW)*, 2020.
- [26] A. Khakifirooz, S. Balasubrahmanyam, R. Fastow, K. H. Gaewsky, C. W. Ha, R. Haque, O. W. Jungroth, S. Law, A. S. Madraswala, B. Ngo, V. Naveen Prabhu, S. Rajwade, K. Ramamurthi, R. S. Shenoy, J. Snyder, C. Sun, D. Thimmegowda, B. M. Pathak, and P. Kalavade, "A 1Tb 4b/Cell 144-Tier Floating-Gate 3D-NAND Flash Memory with 40MB/s Program Throughput and 13.8 Gb/mm² Bit Density," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021.
- [27] B. S. Kim, J. Choi, and S. L. Min, "Design tradeoffs for SSD reliability," in *USENIX Conference on File and Storage Technologies (FAST)*, 2019.
- [28] C. Kim, D.-H. Kim, W. Jeong, H.-J. Kim, I. H. Park, H.-W. Park, J. Lee, J. Park, Y.-L. Ahn, J. Y. Lee, S.-B. Kim, H. Yoon, J. D. Yu, N. Choi, N. Kim, H. Jang, J. Park, S. Song, Y. Park, J. Bang, S. Hong, Y. Choi, M.-S. Kim, H. Kim, P. Kwak, J.-D. Ihm, D. S. Byeon, J.-Y. Lee, K.-T. Park, and K.-H. Kyung, "A 512-Gb 3-b/Cell 64-stacked WL 3-D-NAND flash memory," *IEEE Journal of Solid-State Circuits*, vol. 53, no. 1, 2017.
- [29] D.-H. Kim, H. Kim, S.-W. Yun, Y. Song, J. Kim, S.-M. Joe, K. hwa Kang, J. Jang, H.-J. Yoon, K. Lee, M.-S. Kim, J. Kwon, J.-Y. Jo, S. Park, J. Park, J.-H. Cho, S. Park, G. Kim, J. Bang, H. S. Kim, J.-W. Park, D. Lee, S. H. Lee, S. H. Moon, M.-H. Choi, P. Kwak, J. Kim, J. Kim, K. Jang, I. H. Park, S. H. Moon, M.-H. Choi, P. Kwak, J.-Y. Park, Y. Choi, S. Kim, S. Lee, D.-K. Kang, J.-D. Lim, D.-S. Byeon, K.-W. Song, J. H. Choi, S.-J. Hwang, and J. Jeong, "A 1Tb 4b/cell NAND Flash Memory with tPROG=2ms, tR=110μs and 1.2Gb/s High-Speed IO Rate," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020.
- [30] M. Kim, S. W. Yun, J. Park, H. K. Park, J. Lee, Y. S. Kim, D. Na, S. Choi, Y. Song, J. Lee, H. Yoon, K. Lee, B. Jeong, S. Kim, J. Park, C. A. Lee, J. Lee, J. Lee, J. Y. Chun, J. Jang, Y. Yang, S. H. Moon, M. Choi, W. Kim, J. Kim, S. Yoon, P. Kwak, M. Lee, R. Song, S. Kim, C. Yoon, D. Kang, J.-Y. Lee, and J. Song, "A 1Tb 3b/Cell 8th-Generation 3D-NAND Flash Memory with 164MB/s Write Throughput and a 2.4Gb/s Interface," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [31] M. Kwon, J. Zhang, G. Park, W. Choi, D. Donofrio, J. Shalf, M. Kandemir, and M. Jung, "Tracetracker: Hardware/software co-evaluation for large-scale i/o workload reconstruction," in *IEEE International Symposium on Workload Characterization (IISWC)*, 2017.

- [32] S. Lee, C. Kim, M. Kim, S.-m. Joe, J. Jang, S. Kim, K. Lee, J. Kim, J. Park, H.-J. Lee, M. Kim, S. Lee, S. Lee, J. Bang, D. Shin, H. Jang, D. Lee, N. Kim, J. Jo, J. Park, S. Park, Y. Rho, Y. Park, H.-j. Kim, C. A. Lee, C. Yu, Y. Min, M. Kim, K. Kim, S. Moon, H. Kim, Y. Choi, Y. Ryu, J. Choi, M. Lee, J. Kim, G. S. Choo, J.-D. Lim, D.-S. Byeon, K. Song, K.-T. Park, and K.-h. Kyung, "A 1Tb 4b/cell 64-stacked-WL 3D NAND flash memory with 12MB/s program throughput," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2018.
- [33] J. Li, Q. Wang, P. P. Lee, and C. Shi, "An in-depth comparative analysis of cloud block storage workloads: Findings and implications," *arXiv preprint arXiv:2203.10766*, 2022.
- [34] Q. Li, L. Shi, Y. Di, C. Gao, C. Ji, Y. Liang, and C. J. Xue, "Process variation aware read performance improvement for LDPC-based NAND flash memory," *IEEE Transactions on Reliability*, vol. 69, no. 1, 2019.
- [35] Q. Li, L. Shi, C. J. Xue, K. Wu, C. Ji, Q. Zhuge, and E. H.-M. Sha, "Access characteristic guided read and write cost regulation for performance improvement on flash memory," in *USENIX Conference on File and Storage Technologies (FAST)*, 2016.
- [36] Q. Li, L. Shi, C. J. Xue, Q. Zhuge, and E. H.-M. Sha, "Improving LDPC performance via asymmetric sensing level placement on flash memory," in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2017.
- [37] S. Liang, Z. Qiao, S. Tang, J. Hochstetler, S. Fu, W. Shi, and H.-B. Chen, "An empirical study of quad-level cell (qlc) nand flash ssds for big data applications," in *IEEE International Conference on Big Data (Big Data)*, 2019.
- [38] C.-Y. Liu, J. Kotra, M. Jung, and M. Kandemir, "PEN: Design and evaluation of partial-erase for 3D NAND-based high density SSDs," in *USENIX Conference on File and Storage Technologies (FAST)*, 2018.
- [39] C.-Y. Liu, J. B. Kotra, M. Jung, M. T. Kandemir, and C. R. Das, "SOML Read: Rethinking the read operation granularity of 3D NAND SSDs," in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [40] C.-Y. Liu, Y. Lee, W. Choi, M. Jung, M. T. Kandemir, and C. Das, "GSSA: A resource allocation scheme customized for 3D NAND SSDs," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021.
- [41] C.-Y. Liu, Y. Lee, M. Jung, M. T. Kandemir, and W. Choi, "Prolonging 3D NAND SSD lifetime via read latency relaxation," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [42] J. Liu, S. Chen, T. Wu, and H. Zhang, "A novel hot data identification mechanism for nand flash memory," *IEEE Transactions on Consumer Electronics*, vol. 61, no. 4, 2015.
- [43] S. Liu and X. Zou, "QLC NAND study and enhanced gray coding methods for sixteen-level-based program algorithms," *Microelectronics journal*, vol. 66, 2017.
- [44] Y. Luo, Y. Cai, S. Ghose, J. Choi, and O. Mutlu, "Warm: Improving NAND flash memory lifetime with write-hotness aware retention management," in *IEEE Symposium on Mass Storage Systems and Technologies (MSST)*, 2015.
- [45] Y. Luo, S. Ghose, Y. Cai, E. F. Haratsch, and O. Mutlu, "HeatWatch: Improving 3D NAND flash memory device reliability by exploiting self-recovery and temperature awareness," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018.
- [46] Y. Luo, S. Ghose, Y. Cai, E. F. Haratsch, and O. Mutlu, "Improving 3D NAND flash memory lifetime by tolerating early retention loss and process variation," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.
- [47] Y. Lv, L. Shi, C. J. Xue, Q. Zhuge, and E. H.-M. Sha, "Latency variation aware read performance optimization on 3D high density NAND flash memory," in *ACM Great Lakes Symposium on VLSI (GLSVLSI)*, 2020.
- [48] F. Margaglia, G. Yadgar, E. Yaakobi, Y. Li, A. Schuster, and A. Brinkmann, "The devil is in the details: Implementing flash page reuse with WOM codes," in *USENIX Conference on File and Storage Technologies (FAST)*, 2016.
- [49] D. Narayanan, E. Thereska, A. Donnelly, S. Elnikety, and A. Rowstron, "Migrating server storage to SSDs: analysis of tradeoffs," in *ACM European conference on Computer systems (EuroSys)*, 2009.
- [50] D. T. Nguyen, G. Zhou, G. Xing, X. Qi, Z. Hao, G. Peng, and Q. Yang, "Reducing smartphone application delay through read/write isolation," in *Annual International Conference on Mobile Systems (MobiSys)*, 2015.
- [51] Y. Pan, G. Dong, Q. Wu, and T. Zhang, "Quasi-nonvolatile ssd: Trading flash memory nonvolatility to improve storage system performance for enterprise applications," *IEEE International Symposium on High-Performance Comp Architecture*, 2012.
- [52] N. Papandreou, N. Ioannou, T. Parnell, R. Pletka, M. Stanisavljevic, R. Stoica, S. Tomic, and H. Pozidis, "Reliability of 3D NAND flash memory with a focus on read voltage calibration from a system aspect," in *IEEE Non-Volatile Memory Technology Symposium (NVMTS)*, 2019.
- [53] N. Papandreou, H. Pozidis, N. Ioannou, T. Parnell, R. Pletka, M. Stanisavljevic, R. Stoica, S. Tomic, P. Breen, G. Tressler, A. Fry, T. Fisher, and A. Walls, "Open block characterization and read voltage calibration of 3D QLC NAND flash," in *IEEE International Reliability Physics Symposium (IRPS)*, 2020.
- [54] D. Park and D. H. Du, "Hot data identification for flash-based storage systems using multiple bloom filters," in *IEEE Symposium on Mass Storage Systems and Technologies (MSST)*, 2011.
- [55] J.-W. Park, D. Kim, S. Ok, J. Park, T. Kwon, H. Lee, S. Lim, S.-Y. Jung, H.-J. Choi, T. Kang, G. Park, C. Yang, J.-G. Choi, G. Ko, J.-H. Shin, I. Yang, J. Nam, H. Sohn, S. in Hong, Y. Jeong, S.-W. Choi, C. Choi, H.-S. Shin, J.-Y. Lim, D. Youn, S. Nam, J. Lee, M. H. Ahn, H. Lee, S. Lee, J. Park, K. Gwon, W. Jeong, J. Choi, J. Kim, and K. Jin, "A 176-Stacked 512Gb 3b/Cell 3D-NAND Flash with 10.8 Gb/mm² Density with a Peripheral Circuit Under Cell Array Architecture," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021.
- [56] J. Park, M. Kim, M. Chun, L. Orosa, J. Kim, and O. Mutlu, "Reducing solid-state drive read latency by optimizing read-retry," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021.
- [57] T. Pekny, L. Vu, J. Tsai, D. Srinivasan, E. Yu, J. Pabustan, J. Xu, S. Deshmukh, K.-F. Chan, M. Piccardi, K. Xu, G. Wang, K. Shakeri, V. Patel, T. Iwasaki, T. Wang, P. Musunuri, C. Gu, A. Mohammadzadeh, A. Ghalam, V. Moschiano, T. Vali, J. Park, J. Lee, and R. Ghodsi, "A 1-Tb Density 4b/Cell 3D-NAND Flash on 176-Tier Technology with 4-Independent Planes for Read using CMOS-Under-the-Array," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [58] R. Pletka, I. Koltsidas, N. Ioannou, S. Tomić, N. Papandreou, T. Parnell, H. Pozidis, A. Fry, and T. Fisher, "Management of next-generation NAND flash to achieve enterprise-level endurance and latency targets," *ACM Transactions on Storage (TOS)*, vol. 14, no. 4, 2018.
- [59] R. Pletka, N. Papandreou, R. Stoica, H. Pozidis, N. Ioannou, T. Fisher, A. Fry, K. Ingram, and A. Walls, "Improving NAND flash performance with read heat separation," in *International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2020.
- [60] N. Shibata, K. Kanda, T. Shimizu, J. Nakai, O. Nagao, N. Kobayashi, M. Miakashi, Y. Nagadomi, T. Nakano, T. Kawabe, T. Shibuya, M. Sako, K. Yanagidaira, T. Hashimoto, H. Date, M. Sato, T. Nakagawa, H. Takamoto, J. Musha, T. Minamoto, M. Uda, D. Nakamura, K. Sakurai, T. Yamashita, J. Zhou, R. Tachibana, T. Takagiwa, T. Sugimoto, M. Ogawa, Y. Ochi, K. Kawaguchi, M. Kojima, T. Ogawa, T. Hashiguchi, R. Fukuda, M. Masuda, K. Kawakami, T. Someya, Y. Kajitani, Y. Matsumoto, N. Morozumi, J. Sato, N. Raghunathan, Y. L. Koh, S. Chen, J. Lee, H. Nasu, H. Sugawara, K. Hosono, T. Hisada, T. Kaneko, and H. Nakamura, "A 1.33-Tb 4-Bit/Cell 3-D Flash Memory on a 96-Word-Line-Layer Technology," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019.
- [61] Y. Shim, M. Kim, M. Chun, J. Park, Y. Kim, and J. Kim, "Exploiting process similarity of 3D flash memory for high performance SSDs," in *Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.
- [62] K.-D. Suh, B.-H. Suh, Y.-H. Lim, J.-K. Kim, Y.-J. Choi, Y.-N. Koh, S.-S. Lee, S.-C. Kwon, B.-S. Choi, J.-S. Yum, J.-H. Choi, J.-R. Kim, and H.-K. Lim, "A 3.3v 32mb nand flash memory with incremental step pulse programming scheme," *IEEE Journal of Solid-State Circuits*, vol. 30, no. 11, 1995.
- [63] Toshiba, "Meeting application needs with a new generation of advanced storage," 2019.
- [64] F. Wu, M. Zhang, Y. Du, W. Liu, Z. Lu, J. Wan, Z. Tan, and C. Xie, "Using error modes aware LDPC to improve decoding performance of 3-D TLC NAND flash," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 39, no. 4, 2019.
- [65] G. Yadgar, M. Gabel, S. Jaffer, and B. Schroeder, "YCSB RocksDB SSD traces (SNIA IOTTA trace 28571)," in *SNIA IOTTA Trace Repository*, Dec. 2018. [Online]. Available: <http://iota.snia.org/traces/block-io/28568?only=28571>

- [66] G. Yadgar, M. Gabel, S. Jaffer, and B. Schroeder, "SSD-based workload characteristics and their performance implications," *ACM Transactions on Storage*, vol. 17, pp. 1–26, 01 2021.
- [67] G. Yadgar, E. Yaakobi, and A. Schuster, "Write Once, Get 50% Free: Saving SSD Erase Costs Using WOM Codes," in *USENIX Conference on File and Storage Technologies (FAST)*, 2015.
- [68] M.-C. Yang, Y.-M. Chang, C.-W. Tsao, P.-C. Huang, Y.-H. Chang, and T.-W. Kuo, "Garbage collection and wear leveling for flash memory: Past and future," in *International Conference on Smart Computing*, 2014.
- [69] Y. Yang, V. Misra, and D. Rubenstein, "On the optimality of greedy garbage collection for ssds," *SIGMETRICS Perform. Eval. Rev.*, 2015.
- [70] J. Yuh, J. Li, H. Li, Y. Oyama, C. Hsu, P. Anantula, S. Jeong, A. Amarnath, S. Darne, S. Bhatia, T. Tang, A. Arya, N. Rastogi, N. Ookuma, H. Mizukoshi, A. Yap, D. Wang, S. Kim, Y. Wu, M. Peng, J. Lu, T. Ip, S. Malhotra, D. Han, M. Okumura, J. Liu, J. Sohn, H. Chibvongodze, M. Balaga, A. Matsuda, C. Puri, C. Chen, I. K. V, C. G, V. Ramachandra, Y. Kato, R. Kumar, H. Wang, F. Moogat, I.-S. Yoon, K. Kanda, T. Shimizu, N. Shibata, T. Shigeoka, K. Yanagidaira, T. Kodama, R. Fukuda, Y. Hirashima, and M. Abe, "A 1-Tb 4b/Cell 4-Plane 162-Layer 3D Flash Memory With a 2.4-Gb/s I/O Speed Interface," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [71] K. Zhao, W. Zhao, H. Sun, X. Zhang, N. Zheng, and T. Zhang, "LDPC-in-SSD: Making advanced error correction codes work effectively in solid state drives," in *USENIX Conference on File and Storage Technologies (FAST)*, 2013.